

Database Systems Lab

REST

Christian Rauch

(changed: August 24, 2025)

Representational State Transfer

REST is an architectural pattern for stateless client-server communication. It is simple, scalable, and based on HTTP and related web standards. Everything is treated as a resource identified by a URI.

```
1 GET /api/products
2 GET /api/products?category=shoes&brand=adidas
3 GET /api/products/280030
4 GET /api/users/u3013
5 GET /api/users/u3013/orders
6 GET /api/users/u3013/orders/43209-339/shipping/4
```

HTTP methods are used to indicate the operation to apply.

Read	→	GET
Create	→	POST
Update (partially)	→	PUT (PATCH)
Delete	→	DELETE

RESTful services are often prefixed with /api.

Read, Create, Update, Delete

The HTTP request header contains the request line, the host, and further meta data (e.g., user-agent: ..., accept: application/json).

```
1 POST /api/products HTTP/1.1
2 host: 127.0.0.1:8080
3 Content-Type: application/json
```

The HTTP request body contains the payload (usually JSON).

```
1 {
2   "name": "Converse Chucks classic",
3   "price": 59.0,
4   "categories": ["sneakers", "streetwear"]
5 }
```

POST, PUT, and PATCH are similar. GET and DELETE often need no body.

```
1 GET /api/products/28003
2 DELETE /api/users/u3013/orders/43209-339
```

Backend Request Handling

```
1 @app.route('/api/products/', methods=['POST'])
2 def add_product():
3     data = request.get_json()
4     ps = ProductService()
5     product = ps.create_product(
6         session['usr'], data['name'], data['price'])
7     return jsonify(product), 201
8
9 @app.route('/api/orders/<oid>',
10            methods=['GET', 'DELETE'])
11 def get_order(oid: int):
12     ps = ProductService()
13     if request.method == 'GET':
14         order = ps.get_order(session['usr'], oid)
15         return jsonify(order), 200
16     else:
17         ps.delete_order(session['usr'], oid)
18         return jsonify({'deleted': oid}), 200
```

Frontend Response Handling

```
1  async function addProduct(name, price) {
2      const response = await fetch(
3          '/api/products/', {
4              method: 'POST',
5              headers: {
6                  'Content-Type': 'application/json'
7              },
8              body: JSON.stringify({
9                  name: name, price: price
10             })
11         });
12
13     if (!response.ok) { ... return; }
14
15     const product = await response.json();
16     console.log('Product added:', product);
17     ...
18 }
```

Conclusion

Thank you for your attention!